UNIDAD 5
FUNCIONES

Por Félix Ferreiras
y Zobiesky Ovalle

Funciones en C++

Objetivos

En esta unidad usted aprenderá a:

- Utilizar funciones en el desarrollo de algoritmos entendiendo su importancia para la solución de problemas
- Trabajar con distintos tipos de argumentos y valor de retorno en las funciones
- Crear funciones sobrecargadas para proveer distintas firmas de función que hagan mas versátil su ejecución
- Trabajar con argumentos de línea de comando
- Crear librerías para potencializar la reutilización del código

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

¿Que es una función?

Ventajas de las funciones

Funciones: concepto

- Una función es una porción de código que ejecuta una tarea especifica
- Esta porción de código tiene un nombre el cual es referido para llamar la función y ejecutar la tarea para la que fue construida
- El control del programa es transferido a la función mientras esta se encuentra en ejecución

Funciones: concepto

- La función puede requerir o no que le envíen valores cuando es invocada
- La función puede retornar o no algún valor
- La labor de retornar algún valor en una función es realizada por la sentencia **return** el cual debe devolver un valor del mismo tipo que el valor de retorno especificado en la declaración de la función
- Cuando el flujo del programa dentro de una función alcanza la sentencia **return** el flujo del programa es retornado a la función llamadora

Funciones: ventajas

- Las funciones proveen reutilización de código y reducen su duplicidad
- Permiten mejor lectura del código fuente
- Facilitan el ocultamiento de información
- Permiten descomponer un programa en módulos mas pequeños durante su desarrollo, esto es, reducir la complejidad de un problema al dividirlo en varios problemas o piezas de código que sean más manejables

Funciones: divide y conquistarás

```
int main(int argc, char**argv)
{
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
    sentencia;
}
```

```
int main(int argc, char**argv)
{
    sentencia;
    sentencia;
}
```

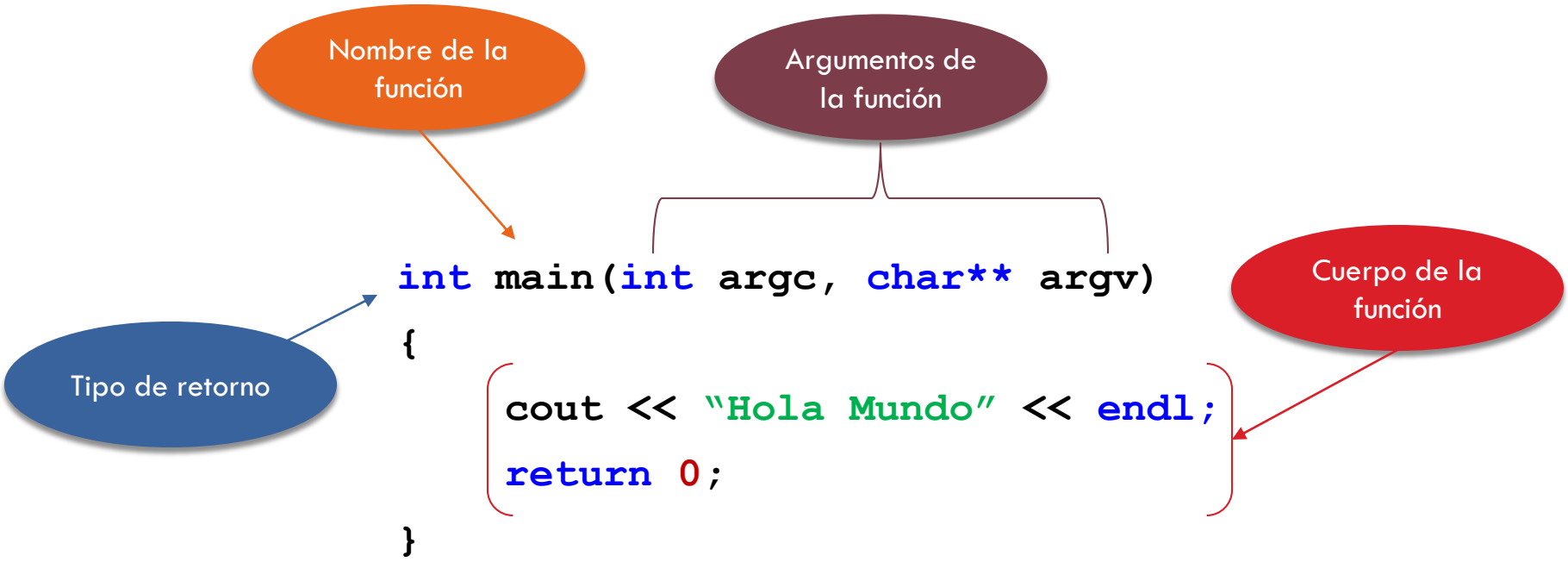
```
void funcion1()
{
    sentencia;
    sentencia;
}
```

```
int funcion2()
{
    sentencia;
    sentencia;
}
```

Cómo definir una función

- Para definir una función debemos tomar en cuenta los siguientes factores
  - Si retornará o no algún valor: tipo de dato de retorno
  - Si requerirá o no que se le envíen valores cuando sea invocada: argumentos
  - Su nombre: debe respetar las reglas de los identificadores
  - La tarea que realizará: cuerpo de la función

Cómo definir una función



Cómo definir una función

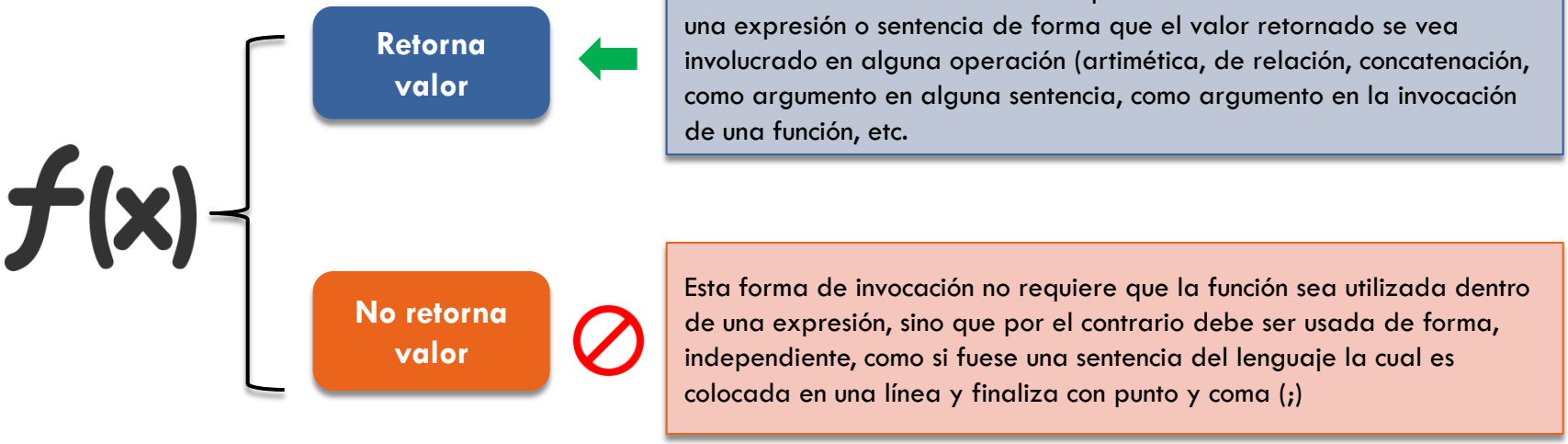
- Definiciones
  - ***Tipo de retorno***: el tipo de retorno se refiere al tipo de dato que la función podrá retornar cuando ésta sea invocada, como ejemplo de lo que puede ser colocado como tipo de retorno tenemos: int, double, bool, short, string, char, punteros, vectores, estructuras. Es importante notar que no necesariamente las funciones debe retornar algún valor, en cuyo caso el tipo de dato especificado debe ser **void** significando esto que la función NO devuelve valor alguno
  - ***Nombre***: se refiere al nombre que tendrá la función el cual será el referido cuando ésta sea invocada por otra función, el nombre que se elija para la función debe ser sugerente a la tarea que la misma realiza respetando las reglas de los identificadores
  - ***Argumentos o parametros***: se refiere a la lista de los valores que una función puede recibir, este listado debe ir separado por coma y cada argumento debe tener tipo y nombre de forma que pueda ser referido dentro de la función
  - ***Cuerpo***: es la tarea que realiza la función, las sentencias que la forman

Cómo llamar a una función

- Para llamar una función debemos tomar en cuenta los siguientes factores:
  - Si la función retorna o no un valor
  - Conocer los argumentos que la función recibe

- Es muy importante tomar en cuenta que si la función retorna algún valor, ésta debe ser invocada dentro de algún contexto o expresión de forma que el valor retornado pueda realizar alguna tarea, ya sea asignarlo a una variable, usarlo en una expresión aritmética, de relacion, incluso hasta ser usado como argumento en el llamado de otra función

Cómo llamar a una función



Definiendo y llamando a una función

```
1   #include <iostream>
2
3   using namespace std;
4
5   void despliegaMensaje()
6   {
7       cout << "Bienvenido a nuestra prueba\n";
8   }
9
10  int suma(int a, int b)
11  {
12      return a + b;
13  }
14
15  int main(int argc, char** argv)
16  {
17      despliegaMensaje();
18      cout << suma(4, 15) << endl;
19      return 0;
20  }
```

La siguiente porción de código define una función llamada ***despliegaMensaje*** la cual no recibe argumentos y cuya tarea consiste en imprimir un mensaje de bienvenida

Este segundo bloque define una función llamada ***suma*** la cual recibe como argumento dos valores enteros llamados ***a*** y ***b*** la cual tiene la tarea de sumar ambos argumentos y retornar su resultado, por eso esta función tiene a la izquierda de su nombre la palabra reservada ***int*** la cual indica que la función retorna un valor entero tipo ***int***

En este ámbito se produce el llamado a la función ***despliegaMensaje***, como se puede observar dicha funciona no retorna valor, es decir, es tipo ***void***, por lo que su invocación es realizada en una línea única

Luego llamamos la función ***suma*** a la cual le pasamos dos números los cuales serán recibidos para utilizarlos en el cálculo de la suma, el hecho de invocar la función implica que el control del programa se pasa a la función la cual una vez finalice su ejecución retorna el resultado de la suma el cual es utilizado por el ***cout*** para desplegarlo en pantalla. De ahí la razón por la cual las funciones que retornan valor deben ser invocadas desde alguna expresión o algún contexto con el cual hacer algo con lo que devuelve la función llamada

La firma de la función

- La firma de la función es lo que viene a identificarla de forma única dentro de un programa o librería de tal manera que al ser invocada no exista exactamente otra función con las mismas especifcaciones
- ¿Qué se toma en cuenta para determinar que una función es única o más bien su firma?
  - El tipo de retorno: que tipo de dato retorna la función
  - El nombre de la función
  - La lista de argumentos o parámetros

Prototipo de una función

- Cómo se vió en el ejemplo anterior, las funciones necesitaron ser declaradas antes de la función ***main*** esto es debido a para poder invocarlas, el programa necesita saber que ya existen por lo que deben ir definidas antes a la función principal, de igual forma si una función A llama a otra función B, la función B debe ser definida antes que la que la llama, o sea, antes que la función A

Prototipo de una función

- No es obligatorio tener que escribir las funciones antes de la función ***main***, para esto podemos utilizar los **prototipos** que no es más que simplemente la firma de la función, esto es la función sin cuerpo para luego implementarlas despues de la función ***main*** implicando que debe ser escrita complementamente
- El prototipo de una función es su firma; contiene el nombre de la función, así como sus parámetros con sus tipos y su valor de retorno

Prototipo de una función

```
1   #include <iostream>
2
3   using namespace std;
4
5   /** prototipo de las funciones **/
6   void despliegaMensaje();
7   int suma(int a, int b);
8
9   int main(int argc, char** argv)
10  {
11      despliegaMensaje();
12      cout << suma(4, 15) << endl;
13      return 0;
14  }
15
16  void despliegaMensaje()
17  {
18      cout << "Bienvenido a nuestra prueba\n";
19  }
20
21  int suma(int a, int b)
22  {
23      return a + b;
24  }
```

Como puede observar, en este ejemplo los prototipos son definidos en el área donde se definen las variables globales y luego son implementados al final. Implementar significa escribir la función completa incluyendo su cuerpo, el prototipo solo tiene la cabeza, header o firma de la función.

Los prototipos también pudieron ser escritos de la siguiente forma:

```
void despliegaMensaje();
int suma(int, int);
```

En los prototipos no es necesario especificar el nombre del argumento o parámetro, lo importante es el tipo de dato de cada uno.

Funciones, argumentos y valor de retorno

- Los parámetros así como las variables y constantes locales solo tienen alcance (son visibles) dentro de la función donde son declarados
- El valor retornado por la función (return valor) debe ser del tipo de retorno especificado en la firma y puede ser usado en cualquier expresión
- El tipo de retorno debe ser un tipo ya definido (int, char, bool, string, float, double, una estructura, un vector, un puntero, etc.)

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Argumentos en las funciones

Pase por valor y pase por referencia

Argumentos por defecto

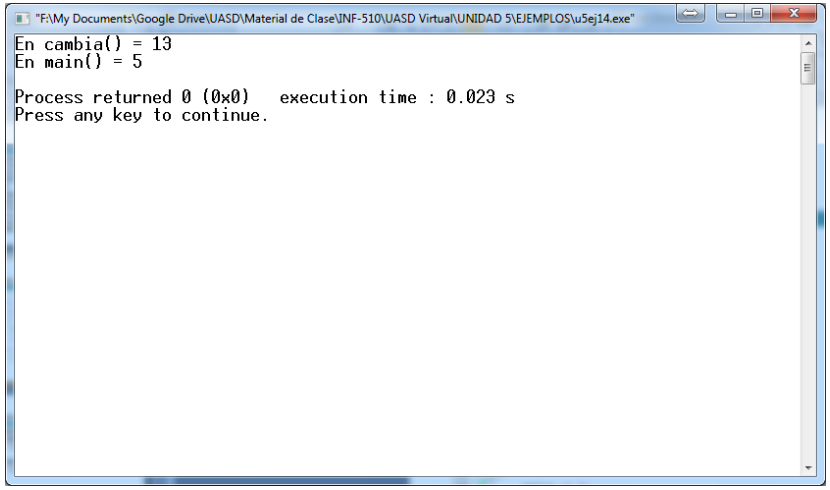
Argumentos de línea de comando

Pase por valor

- Cuando pasamos valores a una función, lo que realmente es transferido es una copia de dicho valor, significando esto que lo que le suceda a los valores pasados dentro de la función no tiene efecto alguno sobre los valores fuera de ella
- Si desea obtener algún resultado que dependa de los argumentos pasados a la función debe especificar un tipo de retorno
- Otra opción es pasar los argumentos por referencia, aunque esto no es lo mismo que retornar un valor, más adelante lo veremos en detalle

Pase por valor

```
1   #include <iostream>
2
3   using namespace std;
4
5   /** Pase por valor **/
6   void cambia(int);
7
8   int main(int argc, char** argv)
9   {
10      int numero = 5;
11      cambia(numero);
12      cout << "En main() = "
13           << numero << endl;
14      return 0;
15  }
16
17  void cambia(int valor)
18  {
19      valor += 8;
20      cout << "En cambia() = "
21           << valor << endl;
22  }
```



Fíjese que dentro de la función ***cambia*** se altera el valor pasado aumentándolo en 8, no obstante esto no tiene ningún efecto en el valor originalmente pasado, esto puede ser apreciado en cada despliegue por pantalla de dicho valor dentro de la función ***cambia*** y dentro de la función ***main***

Pase por referencia

- Cuando pasamos data por referencia a una función permitimos que ésta pueda acceder a los valores originales
- Para esto utilizamos variables de referencia las cuales pueden almacenar la referencia del valor original, es decir, el lugar donde se encuentra dicho valor en la memoria y a través de ellas poder modificar los valores originales
- Las variables de referencia son definidas como argumento en una función utilizando el operador de referencia **&**

Operador de referencia &

- **&** significa “dirección de”
- **&** contiene la dirección de una variable
- Usando **&** podemos encontrar la dirección que ocupa una variable
- El operador de referencia es un sustituto práctico de las direcciones de las variables
- Se comportan como los punteros (lo veremos en la próxima unidad)

Operador de referencia &

- C++ implementa el operador de referencia a bajo nivel como puntero usando lenguaje ensamblador
- Las referencias son ligadas a una variable y luego no pueden ser religadas a otra variable
- El operador de referencia es un operador de direcciones
- Acceder a una variable vía el operador de referencia es lo mismo que manipular la variable

Operador de referencia &

- Declaración:

```
tipo variable;
tipo &varref = variable;
```

- Declara ***varref*** como una referencia a un valor ***tipo***; es la forma correcta para declarar una referencia fuera de una función
- ***varref*** será un alias (nombre alternativo) de ***variable***; lo cual significa que todo cuanto se haga en ***varref*** se reproducirá en ***variable***

Operador de referencia &

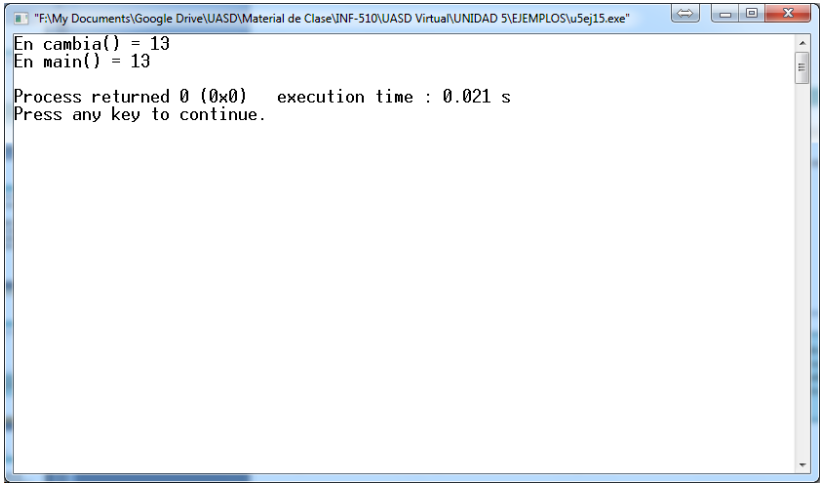
- El significado del operador de referencia depende del contexto en el cual sea usado. Veamos:

`&variable;`	& es un operador de dirección que significa ***"dirección de variable"***; será usado en el tema de punteros
`tipo &varref = variable;`	Declara una referencia ***tipo*** de ***variable***

`tipo funcion(tipo &varref)`
`{`
`  sentencias;`
`}`
	Declara ***varref*** como un parámetro para un pase por referencia en ***funcion***, esto es, acceso a la dirección de la variable enviada como argumento; ***es una referencia a la variable original***, un alias o nombre alternativo de ésta

Pase por referencia

```
1   #include <iostream>
2
3   using namespace std;
4
5   /** Pase por valor **/
6   void cambia(int &);
7
8   int main(int argc, char** argv)
9   {
10      int numero = 5;
11      cambia(numero);
12      cout << "En main() = "
13           << numero << endl;
14      return 0;
15  }
16
17  void cambia(int &valor)
18  {
19      valor += 8;
20      cout << "En cambia() = "
21           << valor << endl;
22  }
```



En este ejemplo podemos observar como el valor de la variable original sí es modificado por la función ***cambia*** esto es recibiendo el valor como una referencia utilizando una variable que pueda manipular el valor original, esto lo hacemos utilizando el símbolo **&** (operador de referencia) el cual indica que la variable a la derecha puede recibir o almacenar una referencia

Pase por referencia

- La función llamadora envía como argumento la dirección de la variable por lo que la función llamada accede así al valor original y no a una copia de éste
- Los cambios realizados en el argumento se realizan también en la variable y hacen que se pierda el valor inicial que fue almacenado antes de llamar a la función permaneciendo el último valor especificado

Argumentos por defecto

- Un argumento por defecto es un valor constante que se le asigna al parámetro en la definición de la función
- El argumento por defecto es usado cuando al llamar la función no se suple el valor correspondiente
- Cuando se llama la función y se le envían argumentos para los parámetros que tienen valores por defecto, éstos últimos son ignorados
- En la lista de parámetros, los argumentos por defecto deben ser los últimos y deben ser declarados secuencialmente; también pueden ser todos los parámetros de la función

Argumentos por defecto

- Veamos los siguientes ejemplos:

```
double areaRectangulo(double = 20.0, double = 15.0);
```

Esta función tiene dos argumentos con valor por defecto, esto significa que si se invoca la función y no se especifica ningun argumento, entonces ambos parámetros tendrán el valor que fue definido

```
void muestra(string, int = 10, bool = true);
```

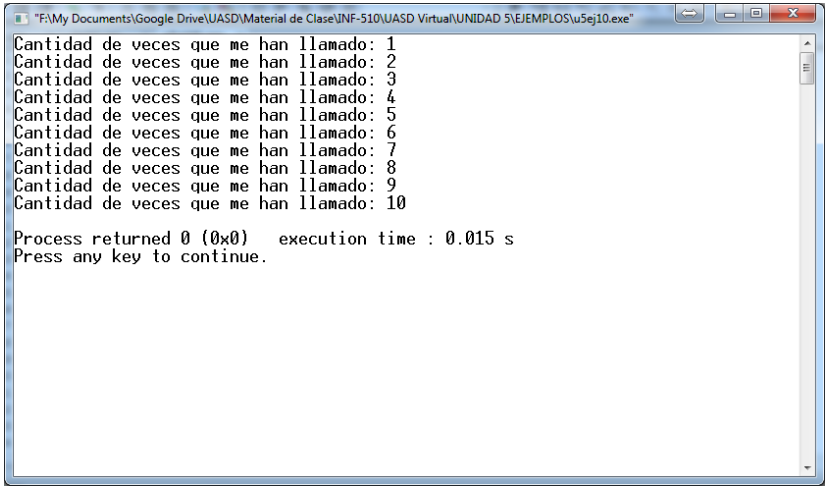
En este ejemplo vemos que sólo los dos últimos argumentos poseen un valor por defecto definido, esto quiere decir que es mandatorio proveer algún valor para el primer argumento

Variables estáticas locales

- Si una función es llamada más de una vez en un programa, cada vez que la función es ejecutada las variables locales definidas en ella son reiniciados al valor que le es asignado en el cuerpo de dicha función
- Esto significa que la variable no es persistente debido a que las variables locales son destruidas al finalizar la ejecución de la función
- Si deseamos agregar persistencia a alguna variable local dentro de una función solo debemos declararla utilizando la palabra reservada **static**
- Declarando la variable como estática permite que la variable solo sea creada una vez

Variables estáticas locales

```
1   #include <iostream>
2
3   using namespace std;
4
5   void printMessage();
6
7   int main(int argc, char** argv)
8   {
9       for (int i = 0; i < 10; i++)
10      {
11          printMessage();
12      }
13
14      return 0;
15  }
16
17  void printMessage()
18  {
19      static int cantidadDeLlamadas;
20      cantidadDeLlamadas++;
21      cout << "Cantidad de veces que me han llamado: "
22           << cantidadDeLlamadas << endl;
23  }
```



Como pueden observar la variable ***cantidadDeLlamadas*** no es destruida al salir de la función, sino que mantiene su valor a través de las diversas invocaciones realizadas

IMPORTANTE
Las variables estáticas al ser declaradas no tienen valores basura, sino que si son numéricas su valor inicial es cero

Argumentos de línea de comando

- Los argumentos de línea de comando permiten al programa recibir valores desde su ejecución y utilizar dichos argumentos para conocer la intención sobre la utilización del programa por el usuario en cuanto a opciones y funcionalidades predefinidas
- Por supuesto, estas funcionalidades o comportamiento de la aplicación debe ser definida por el programador
- Los argumentos de línea de comando son recibidos por la función que es el punto de arranque de todo programa C++, la función ***main***

Argumentos de línea de comando

- Examinemos la firma completa de la función ***main*** que nos sirve para recibir valores como argumentos de línea de comando (puede ser escrita de dos formas):
  - `int main(int argc, char** argv)`
  - `int main(int argc, char* argv[])`

- Es importante notar que si no nos interesa recibir argumentos de línea de comando la firma de la función ***main*** podría ser escrita de la siguiente forma:
  - `int main()`
  - `int main(void)`

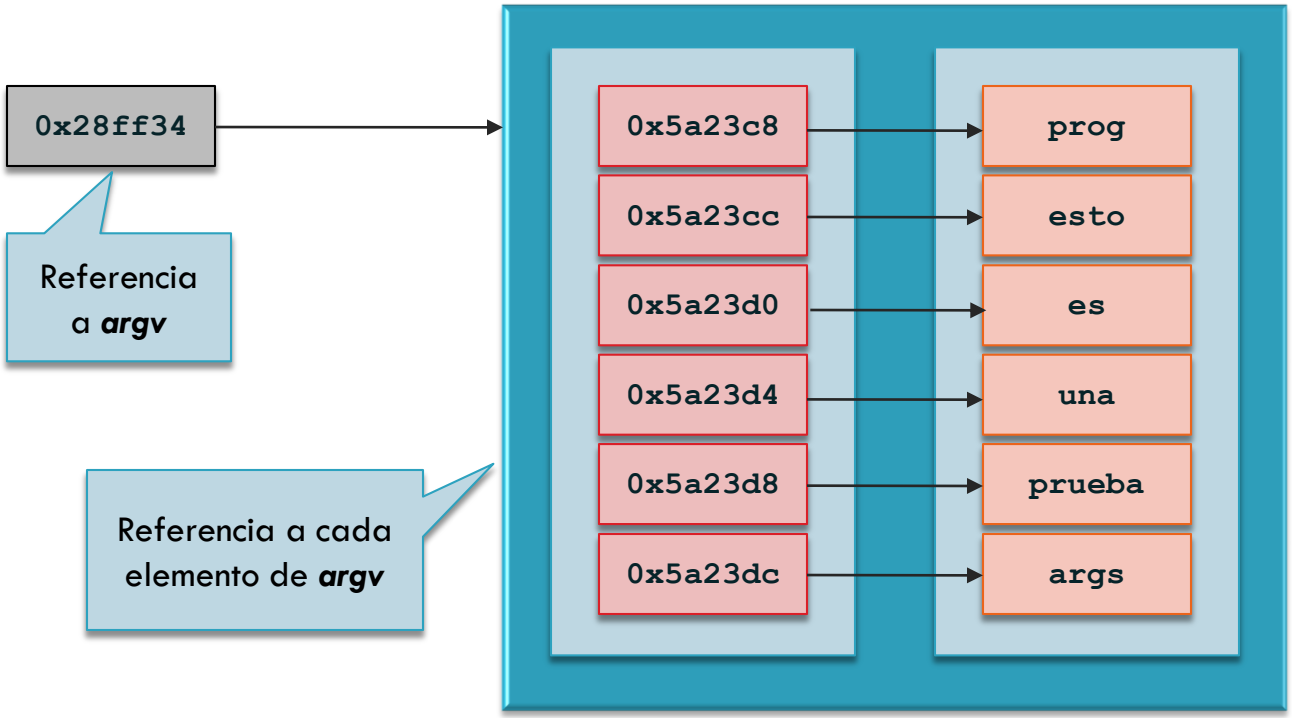
Argumentos de línea de comando

Expresión	Descripción o significado
`int main`	Es el nombre de la función y el tipo de retorno, esto es, la función ***main*** retorna un dato tipo entero (int), esto puede ser usado para devolver algún valor de error si en la ejecución del programa surgió un problema o cero si fue ejecutado exitosamente quien la invoca
`int argc`	Esto define la cantidad de argumentos que recibe la función main. IMPORTANTE: la cantidad de argumentos se determina a través de un caracter separador el cual es el espacio en blanco. En C++ el mismo nombre del programa ejecutado cuenta como un argumento
`char** argv`	Variable que contiene la lista de argumentos recibidos, es un arreglo de constantes de arreglo de ***char*** o un arreglo de un arreglo de ***char***. También podriamos referirlo como un arreglo de cadenas C-Strings o cadenas estilo C (es una referencia de puntero el cual en cada posición del puntero refierea otro puntero, entenderemos este concepto mejor en la próxima unidad)
`char* argv[]`	Es otra forma de escribir la expresión anterior (los arreglos son referencias de constantes de puntero)

Argumentos de línea de comando

Siendo prog.cpp un programa para mostrar los argumentos pasados por línea de comando, ejecutamos:

```
C:\> prog esto es una prueba args
```



Argumentos de línea de comando

- Veamos el contenido del programa:

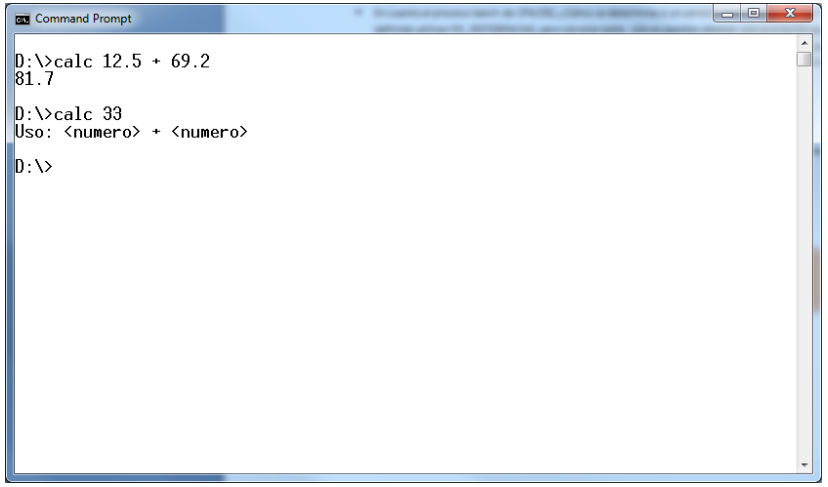
```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      cout << "Direccion argv: "
8           << &argv << endl << endl;
9
10     for (int i = 0; i < argc; i++)
11     {
12         cout << "Argumento [" << i << "] ("
13              << &argv[i] << "): " << argv[i]
14              << endl;
15     }
16
17     return 0;
18 }
```

```
Direccion argv: 0x28ff34

Argumento [0] (0x5e23c8): prog
Argumento [1] (0x5e23cc): esto
Argumento [2] (0x5e23d0): es
Argumento [3] (0x5e23d4): una
Argumento [4] (0x5e23d8): prueba
Argumento [5] (0x5e23dc): args
```

Argumentos de línea de comando: Ejemplo práctico

```
1  #include <iostream>
2  #include <cstdlib>
3
4  using namespace std;
5
6  int main(int argc, char** argv)
7  {
8      const int CANT_ARGS_PERMITIDOS = 4;
9      const string SUMA = "+";
10     double num1 = 0;
11     double num2 = 0;
12
13     if (argc == CANT_ARGS_PERMITIDOS
14         && argv[2] == SUMA)
15     {
16         num1 = atof(argv[1]);
17         num2 = atof(argv[3]);
18
19         cout << num1 + num2 << endl;
20     }
21     else
22     {
23         cout << "Uso: <numero> + <numero>"
24              << endl;
25     }
26
27     return 0;
28 }
```



Este ejemplo hace uso de argumentos de línea de comando para recibir dos operándoos y un operador (+ suma) los cuales serán sumados y su resultado desplegado en pantalla.

Es importante destacar que los elementos del arreglo ***argv*** son de tipo C-String, de ahí el uso de la función ***atof*** para poder convertirlo a ***double*** y así sumarlos.

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Sobrecarga de funciones

Funciones con varias implementaciones

Sobrecarga de funciones

- En C++ al igual que otros lenguajes es posible tener distintas implementaciones para una misma función pero con firma distinta
- Recordemos que la firma de la función es definida por el tipo de retorno, el nombre y los argumentos que recibe
- Esto significa que podemos tener varias funciones con el mismo nombre pero con un listado distinto de parametros y/o un tipo de retorno diferente
- En cada función sobrecargada es posible tener un cuerpo de codigo distinto para cada una, esta es la razon para sobrecargar una función, o tambien soportar otro tipo de parametros en su firma

Sobrecarga de funciones

```cpp
#include <iostream>

using namespace std;

int suma(int, int);
int suma(int [], int);

int main(int argc, char** argv)
{
    int arreglo[5] = {23, 4, 56, 1, -9};

    cout << suma(4, 5) << endl;
    cout << suma(arreglo, 5) << endl;

    return 0;
}

int suma(int a, int b)
{
    return a + b;
}

int suma(int arr[], int cantidad)
{
    int total = 0;
    for (int i = 0; i < cantidad; i++)
        total += arr[i];
    return total;
}
```

Tomemos el siguiente ejemplo en el que tenemos una funcion sobrecargada, veamos las firmas:

```cpp
int suma(int, int);
int suma(int [], int);
```

La primera función recibe dos enteros y devuelve entero, y la segunda función recibe un arreglo de enteros y un entero devolviendo también un mismo tipo de dato. Si observa la implementación de cada funcion, ambas realizan la misma tarea pero tienen un proceso interno distinto.

De ahí la importancia de la sobrecarga de funciones, nos permiten ofrecer distintas implementaciones ocultando los detalles de sus procesos y brindando a quien la consume la flexibilidad de usar la funcion con diferentes parámetros de acuerdo a la necesidad del programador.

Sobrecarga de funciones

- Dos funciones pueden tener el mismo nombre siempre y cuando tengan distinta firma en relación a sus parámetros y/o valor de retorno
- Esta diferencia en sus parámetros puede ser en cantidad o variación, es decir, dos funciones con el mismo nombre pueden tener la misma cantidad de parámetros pero deben diferir en el tipo de dato de al menos uno de ellos
- Dos funciones pueden tener el mismo nombre pero una cantidad distinta de parámetros

Sobrecarga de funciones

```
1  #include <iostream>
2
3  using namespace std;
4
5  int suma(int, int);
6  int suma(int, int, int, int);
7  float suma(float, float);
8
9  int main(int argc, char** argv)
10 {
11     cout << suma(3, 4) << endl;
12     cout << suma(8, 7, 92, 1) << endl;
13     cout << suma(3.2f, 4.5f) << endl;
14     return 0;
15 }
16
17 int suma(int a, int b)
18 {
19     return a + b;
20 }
21
22 int suma(int a, int b, int c, int d)
23 {
24     return a + b + c + d;
25 }
26
27 float suma(float a, float b)
28 {
29     return a + b;
30 }
```

En este ejemplo tenemos tres firmas distintas, una difieren e en cantidad y otra en los tipos:

```
int suma(int, int);
int suma(int, int, int, int);
float suma(float, float);
```

Cuando llamamos a una función lo que se compara es su firma, de esta forma el compilador puede determinar cual función debe ejecutar, se podría decir que la firma de la función viene a formar su nombre completo.

Otro aspecto interesante que podemos ver en el ejemplo es que invocamos la función suma, aquella que esta formada por dos parámetros tipo ***float*** especificando dos literales del mismo tipo, pero para esto debemos colocar una ***f*** al final del número pues un literal de punto flotante es por defecto tipo ***double*** y no tenemos una función sobrecargada que acepte dicho tipo de datos, de ahí que debemos proveer como argumento un conjunto de datos que pueda hacer relación con al menos una de las funciones.

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Funciones y estructuras

Funciones como miembros de estructuras, estructuras como argumentos y funciones que retornan estructuras

Funciones como parte de una estructura

- Las estructuras pueden tener además de los miembros dato que ya aprendimos a manejar, miembros tipo función
- Las funciones miembro, dentro del cuerpo de una estructura, tienen acceso directo a los miembros dato de la misma, esto es, sin utilizar el operador punto
- Dentro de la estructura solo se definen los prototipos de las funciones miembro. La implementación de la función se realiza fuera de la estructura pero haciendo referencia a la estructura a la cual pertenece

Funciones como parte de una estructura

- Sintaxis definición estructura

```
struct IdEstructura
{
    tipo dato_miembro1;
    tipo dato_miembro2;
    .
    .
    .
    tipo funcion_miembro1(args);
    tipo funcion_miembro2(args);
};
```

Concepto

La estructura es definida tal cual lo habíamos hecho en otra unidad, lo que se agrega son nuevos miembros los cuales son funciones. Estas funciones se escriben de la misma forma como se ha definido con anterioridad en esta misma unidad, debe tomar en cuenta su firma ya que también puede aplicar sobrecarga y de ahí dependerá su forma de invocación.

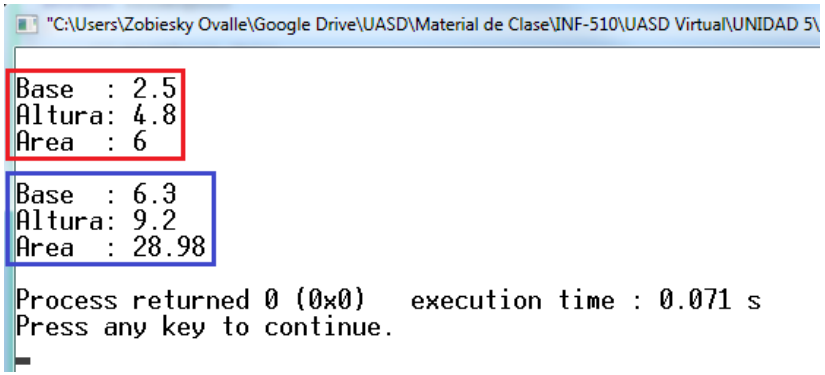
- Sintaxis para implementar de la función miembro

```
tipo idEstructura::funcion(args)
{
      sentencias;
}
```

Funciones como parte de una estructura

```
1  #include <iostream>
2
3  using namespace std;
4
5  struct Triangulo
6  {
7      // miembros dato
8      double base;
9      double altura;
10     // miembros funcion
11     double area();
12 };
13
14 void imprimeTriangulo(Triangulo);
15
16 int main(int argc, char** argv)
17 {
18     Triangulo tri1 = {2.5, 4.8};
19     Triangulo tri2 = {6.3, 9.2};
20     imprimeTriangulo(tri1);
21     imprimeTriangulo(tri2);
22     return 0;
23 }
24
25 double Triangulo::area()
26 {
27     return base * altura / 2;
28 }
```

```
30 void imprimeTriangulo(Triangulo t)
31 {
32     cout << "\nBase  : " << t.base;
33     cout << "\nAltura: " << t.altura;
34     cout << "\nArea  : " << t.area() << endl;
35 }
```



La estructura ***Triangulo*** tiene un miembro función el cual es implementado después de la función ***main*** utilizando la sintaxis mencionada anteriormente. Note que en la ejecución del programa se muestran los datos de dos triángulos, para cada caso se invoca la función ***imprimeTriangulo*** (no pertenece a la estructura) la cual recibe un dato tipo Triangulo como parámetro y dentro de ella se hace referencia a la función ***area*** que pertenece a la estructura, pudiendo ésta última acceder a los miembros dato de la estructura como argumento.

Estructuras como argumentos en una función y tipo de retorno

- Las estructuras o variables de estructuras pueden también ser pasadas como argumento a una función
- Los miembros de la estructura son pasados en su totalidad al pasar la estructura misma a una función
- En el ejemplo anterior se puede apreciar como una función recibe una estructura
- Una función también puede retornar una función como tipo de retorno, esto significa que al invocar la función debe haber una variable del mismo tipo de estructura que la reciba o que la invocación de la función quede en una expresión que requiera dicho tipo de dato

Estructuras como argumentos en una función y tipo de retorno

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

struct Triangulo
{
    double base;
    double altura;

    double area();
};

void imprimeTriangulo(Triangulo);
Triangulo creaTriangulo(double, double);
Triangulo creaTriangulo();

int main(int argc, char** argv)
{
    Triangulo tri1 = creaTriangulo(2.5, 4.8);
    Triangulo tri2 = creaTriangulo();
    imprimeTriangulo(tri1);
    imprimeTriangulo(tri2);
    return 0;
}
```

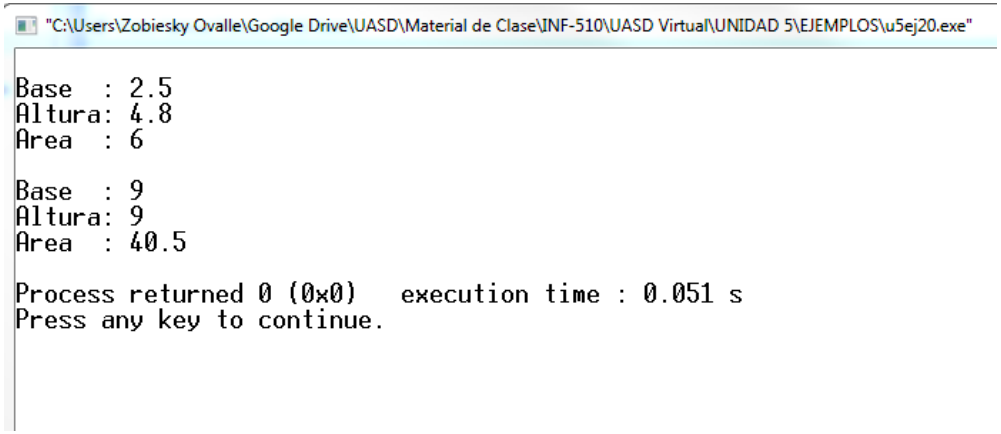
```cpp
double Triangulo::area()
{
    return base * altura / 2;
}

void imprimeTriangulo(Triangulo t)
{
    cout << "\nBase  : " << t.base;
    cout << "\nAltura: " << t.altura;
    cout << "\nArea  : " << t.area() << endl;
}

Triangulo creaTriangulo()
{
    srand(time(0));
    double base = rand() % 10 + 1;
    double altura = rand() % 15 + 1;
    Triangulo t = {base, altura};
    return t;
}

Triangulo creaTriangulo(double base, double altura)
{
    Triangulo t = {base, altura};
    return t;
}
```

Estructuras como argumentos en una función y tipo de retorno



En el ejemplo anterior hay una función llamada ***imprimeTriangulo*** la cual recibe una estructura, este argumento tipo ***Triangulo*** se convierte en una variable local a la función que la recibe pudiendo acceder a todos sus miembros desde la misma función.

```
void imprimeTriangulo(Triangulo);
```

También hay dos funciones sobrecargadas, ambas retornan un dato tipo ***Triangulo***, la diferencia es que una de ellas recibe dos variables tipo ***double*** para usarlas como la base y la altura del triangulo que será creado mientras que la otra función ***creaTriangulo*** genera los valores usados (base y altura) de forma aleatoria.

```
Triangulo creaTriangulo(double, double);
Triangulo creaTriangulo();
```

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Funciones y arreglos

Arreglos como argumentos de funciones

Arreglos como argumentos en funciones

- Hemos visto en algunos ejemplos anteriores que podemos pasar como parámetros arreglos entre funciones
- Para utilizar arreglos como argumentos debemos saber declarar la firma de la función de forma adecuada
- Veamos la siguiente sintaxis para una firma de una función que recibe un arreglo:

```
void imprimeArreglo(int arreglo[], int);
```

El primer argumento representa el arreglo mientras que el segundo es la cantidad de elementos que tiene dicho arreglo

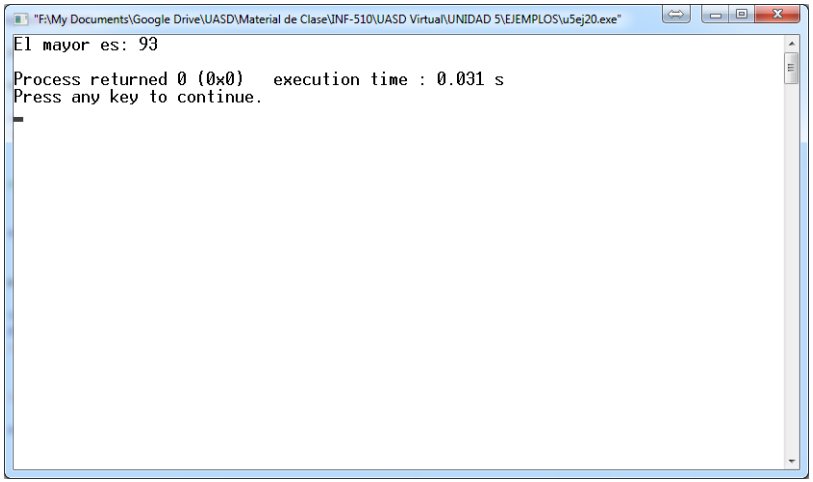
Arreglos como argumentos en funciones

```
1   #include <iostream>
2
3   using namespace std;
4
5   int mayor(int[], int);
6
7   int main(int argc, char** argv)
8   {
9       const int CANTIDAD = 6;
10      int numeros[CANTIDAD]
11            = {25, 9, 93, 33, 28, 55};
12
13      cout << "El mayor es: "
14           << mayor(numeros, CANTIDAD)
15           << endl;
16      return 0;
17  }
18
19  int mayor(int arreglo[], int cantidad)
20  {
21      int mayor = arreglo[0];
22      for (int i = 0; i < cantidad; i++)
23      {
24          if (arreglo[i] > mayor)
25              mayor = arreglo[i];
26      }
27
28      return mayor;
29  }
```

En este programa tenemos una función que recibe un arreglo de enteros y retorna el elemento de mayor valor. Veamos la firma de la función:

```
int mayor(int arreglo[], int cantidad);
```

En la próxima unidad veremos como podemos retornar una referencia a un arreglo utilizando una función, esto lo hacemos con punteros.



Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Funciones y vectores

Vectores como argumentos en funciones y como tipo de retorno

Vectores como argumentos en funciones

- Los vectores también pueden ser pasados como argumento en una función
- Veamos la siguiente sintaxis para una firma de una función que recibe un vector:

```
void imprimeArreglo(vector<int> arreglo);
```

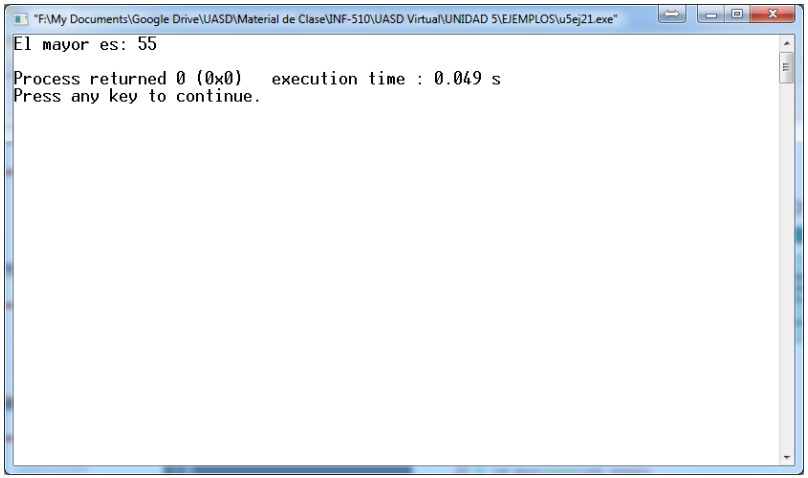
Solo es necesario utilizar un argumento para pasar el vector

Vectores como argumentos en funciones

```
1   #include <iostream>
2   #include <vector>
3
4   using namespace std;
5
6   int mayor(vector<int>);
7
8   int main(int argc, char** argv)
9   {
10      vector<int> numeros;
11      numeros.push_back(34);
12      numeros.push_back(55);
13      numeros.push_back(47);
14
15      cout << "El mayor es: "
16           << mayor(numeros) << endl;
17      return 0;
18  }
19
20  int mayor(vector<int> arreglo)
21  {
22      int mayor = arreglo.at(0);
23      for (int i = 0; i < arreglo.size(); i++)
24      {
25          if (arreglo.at(i) > mayor)
26              mayor = arreglo.at(i);
27      }
28
29      return mayor;
30  }
```

En este programa tenemos una función que recibe un vector de enteros y retorna el elemento de mayor valor. Veamos la firma de la función:

```
int mayor(vector<int> arreglo);
```



Retornando un vector

```
1   #include <iostream>
2   #include <vector>
3   #include <cstdlib>
4   #include <ctime>
5
6   using namespace std;
7
8   vector<int> creaVector(int);
9   void imprimeVector(vector<int>);
10
11  int main(int argc, char** argv)
12  {
13      vector<int> numeros;
14      numeros = creaVector(5);
15      imprimeVector(numeros);
16      return 0;
17  }
```

```
"F:\My Documents\Google Drive\UASD\Material de Clase\INF-510\UASD Virtual\UNIDAD 5\EJEMPLOS\u5ej
38
82
22
65
81

Process returned 0 (0x0)   execution time : 0.151 s
Press any key to continue.
```

```
19  vector<int> creaVector(int cantidad)
20  {
21      srand(time(0));
22      int numero;
23      vector<int> arreglo;
24      for (int i = 0; i < cantidad; i++)
25      {
26          numero = rand() % 100 + 1;
27          arreglo.push_back(numero);
28      }
29
30      return arreglo;
31  }
32
33  void imprimeVector(vector<int> arr)
34  {
35      for (int i = 0; i < arr.size(); i++)
36          cout << arr.at(i) << endl;
37  }
```

En este ejemplo tenemos una función cuya tarea es crear un vector de enteros de la cantidad de elementos que desee el que invoque la función y dar valor a cada elemento con un número aleatorio entre 1 y 100.

En el programa llamamos la función pidiendo crear un vector de 5 elementos y lo asignamos a un vector previamente creado llamado *numeros*.

Contenido

- Que es una función
- Argumentos en las funciones
- Sobrecarga de funciones
- Funciones y estructuras
- Funciones y arreglos
- Funciones y vectores
- Compilación por separado: Librerías

Compilación por separado: Librerías

Modularización del código

Creación de librerías

Compilación por separado

- El potencial real de las funciones es que nos permiten dividir la solución de un problema en la solución de pequeños problemas. Esto es, aplicar el enfoque de "divide y vencerás"
- Cada "problemita" es un módulo y cada módulo es implementado en una función, probado y entregado
- Recordemos que el prototipo de la función le informa al compilador lo que será requerido de la misma
- Un grupo de prototipos relacionados en un archivo independiente es llamado cabecero o ***header***

Compilación por separado

- Los archivos cabeceros o headers son la interface de las funciones
- El código objeto de la implementación de funciones relacionadas se agrupa en ***librerias***
- Las ***librerías*** son una colección de segmentos de código objeto, identificados con un nombre que realizan una tarea específica, usados en forma de módulos para desarrollar programas computacionales
- Cada compilador de C++ viene con una librería estándar y con una librería propietaria

Compilación por separado

- El usuario del compilador también puede construir su propia librería
- Las librerías pueden ser ***estáticas*** o ***dinámicas***. Veremos las estáticas
- Los compiladores disponen de archivos cabeceros que son las interfaces de las librerías que contienen para uso del programador
- Los compiladores C++ ANSI/ISO incorporan estos cabeceros en un espacio de nombre ***std*** permitiendo su uso con la sintaxis **<nombre>** y la sentencia **using namespace std**

Compilación por separado

- El acceso a las funciones se hace por medio del correspondiente ***#include*** en el código fuente del módulo en que se requiera el uso de dichas funciones
- De las funciones contenidas en cada cabecero, todo compilador bien documentado debe incluir explicaciones del uso de cada una de dichas funciones, portabilidad y ejemplos de uso

Creación de una librería

- Pasos para crear una librería (recordar que estamos trabajando con librerías estáticas)
  1. Crear el archivo cabecero o header con los prototipos de las funciones que tendrá (nombre.h)
  2. Crear un archivo por fuente por cada prototipo en el archivo cabecero (funcion.cpp)
  3. Compilar cada funcion por separado: `g++ -c funcion.cpp`
  4. Crear el programa principal que tendrá la referencia a la librería
     - La referencia debe ser **"cabecero.h"**
     - Se utiliza de esta manera para indicar al compilador que se trata de una librería definida por el programador y no una librería del compilador
  5. Compilar: `g++ main.cpp cabecero.h func1.o func2.o -o main`

Creación de una librería

```cpp
#include <iostream>

using namespace std;

int menor(int arreglo[], int cantidad)
{
    int menor = arreglo[0];
    for (int i = 0; i < cantidad; i++)
    {
        if (arreglo[i] < menor)
            menor = arreglo[i];
    }

    return menor;
}
```

menor.cpp

```cpp
#include <iostream>

using namespace std;

int mayor(int arreglo[], int cantidad)
{
    int mayor = arreglo[0];
    for (int i = 0; i < cantidad; i++)
    {
        if (arreglo[i] > mayor)
            mayor = arreglo[i];
    }

    return mayor;
}
```

mayor.cpp

```cpp
#ifndef UTILS_H
#define UTILS_H

    int mayor(int arreglo[], int cantidad);
    int menor(int arreglo[], int cantidad);

#endif
```

utils.h

```cpp
#include <iostream>
#include "utils.h"

using namespace std;

int main(int argc, char** argv)
{
    const int SIZE = 5;
    int arr[SIZE] = {25, 36, 8, 99, 45};

    cout << "El mayor es: "
         << mayor(arr, SIZE) << endl;
    cout << "El menor es: "
         << menor(arr, SIZE) << endl;

    return 0;
}
```

main.cpp

Creación de una librería

- Pasos para crear una librería (recordar que estamos trabajando con librerías estáticas)
  1. `g++ -c mayor.cpp`
  2. `g++ -c menor.cpp`
  3. `g++ main.cpp utils.h mayor.o menor.o -o main`
  4. `main [enter]`

Creación de una librería

- El código de las librerías estáticas es incluido en tiempo de compilación en el archivo ejecutable principal (el que hace referencia a las librerías). Esto provoca que el archivo ejecutable sea más grande en tamaño, no obstante el uso de librerías estáticas no crea dependencia de los modulos que componen la librería pues ya se incluyen en el programa
- Mientras que en las librerías dinámicas o *shared libraries* se reduce el tamaño ocupado por el programa principal, el que hace referencia a las librerías, o cualquier programa que haga referencia a estas evitando duplicidad de código, aunque éstas agregan costo de carga (load) en tiempo de ejecución pues todos los símbolos referidos deben ser validados y conectados

Resumen

- En esta unidad hemos aprendido:
  - Crear funciones con distintos tipos de parámetros como estructuras, arreglos y vectores así como también retornar vectores y estructuras como tipo de dato de retorno en una función
  - Conocer más sobre los prototipos de funciones y como sobrecargarlas (*overload*) alterando su firma
  - Incorporar argumentos por defecto en una función
  - La compilación por separado nos da el beneficio de la modularidad y el potencial para crear librerías que podamos reutilizar en nuestros programas

Resumen

- En esta unidad hemos aprendido:
  - Las funciones nos permiten escribir un código de fácil mantenimiento, nos brindan la posibilidad de trabajar en equipo donde cada módulo de función puede ser delegado a personas distintas
  - Concebir los algoritmos utilizando funciones ofrece la ventaja de minizar la complejidad del problema separándolo en varios problemas menores mejorando el desempeño de la aplicación, creando un código de fácil lectura y más escalable

Bibliografía

- Addison Wesley, Tony Gaddis. (2009). Starting Out with C++ Brief: From Control Structures through Objects. San Francisco, California. 6th Edition.
- Aprende a programar en C++. (2010). Página dedicada a la enseñanza de C++ con libro de libre descarga, recuperado el 3 Junio 2013 de: http://c.conclase.net/curso/index.php
- Página dedicada al lenguaje de programación C++ (2000 – 2014). Tutoriales y recursos de libre descarga, recuperado Enero 2014 en: http://www.cplusplus.com All rights reserved - v3.1
- Paul Deitel, Harvey Deitel (2011). C++ How to Program, New York, 8th Edition, Prentice Hall.



Unidad 5: Funciones

Por Félix Ferreiras y Zobiesky Ovalle